AP3 SEMINARIO DE PRÁCTICA DE INFORMÁTICA - SIGETEC

Sistema Integral de Gestión de Servicios Técnicos

TOMÁS OJEDA
Legajo: VINF015384
Profesor: Hugo Fernando Frías
Fecha de entrega: completar según plataforma

ÍNDICE

1. Introducción y continuidad del proyecto

2. Análisis del modelo de negocios y justificación

3. Aplicación del Proceso Unificado de Desarrollo (PUD)

4. Desarrollo Java: Programación Orientada a Objetos

5. Base de datos MySQL y persistencia

6. Arquitectura, algoritmos y comunicaciones

7. Etapa de pruebas y trazabilidad

8. Ejecución en NetBeans y GitHub

9. Conclusión

10. Referencias

Este informe acompaña el código fuente y los scripts SQL del prototipo. Las evidencias de ejecución local y el enlace al repositorio se agregan después de verificarlos en NetBeans y Workbench.

1. INTRODUCCIÓN Y CONTINUIDAD DEL PROYECTO

1.1. ANTECEDENTES

En el trabajo práctico anterior se definió SIGETEC, Sistema Integral de Gestión de Servicios Técnicos, destinado a una empresa ficticia llamada Nexo Seguridad y Tecnología. La organización realiza instalación y mantenimiento de CCTV, alarmas, cercos eléctricos, redes y reparación informática. Su planificación semanal se administra a través de planillas y comunicaciones que no siempre reúnen el historial completo de cada intervención.

Para esta tercera actividad se desarrolla un prototipo operacional en Java con persistencia en MySQL. Se seleccionó una parte representativa del sistema: registro de clientes, tickets, órdenes de trabajo, técnicos asignados e intervenciones. Este alcance permite verificar componentes reales y continuar el desarrollo incremental planteado con el PUD.

1.2. ALCANCE Y LÍMITES DEL INCREMENTO

Desde: el registro de un cliente y una solicitud de servicio en SIGETEC. Hasta: el cierre de la orden y su ticket, luego de registrar una intervención técnica. El prototipo funciona por consola y no incluye la interfaz web definitiva, login, facturación, guardias, monitoreo en vivo ni inventario.

2. ANÁLISIS DEL MODELO DE NEGOCIOS Y JUSTIFICACIÓN

2.1. NECESIDAD ORGANIZACIONAL

La grilla semanal distribuye técnicos y obras, pero para consultar una falla es importante reconstruir quién la informó, qué se programó, quién realizó la visita y qué solución se aplicó. Por eso el prototipo utiliza identificadores de cliente, ticket y orden, y registra cada intervención dentro de una base de datos relacional.

2.2. OBJETIVO GENERAL DEL PROYECTO

Lograr un prototipo operacional de SIGETEC que demuestre la viabilidad de centralizar el registro, la asignación y el seguimiento de servicios técnicos mediante una solución consistente y verificable.

2.3. OBJETIVOS ESPECÍFICOS DEL PROYECTO

1. Contar con una representación coherente del circuito de atención técnica.
2. Disponer de un prototipo que permita registrar y consultar clientes, tickets y órdenes.
3. Conservar información de forma persistente, relacionada y sin duplicaciones evitables.
4. Demostrar el uso de programación orientada a objetos para diferenciar tipos de solicitudes.
5. Comprobar mediante pruebas el comportamiento del modelo y, posteriormente, las operaciones reales sobre MySQL.

2.4. OBJETIVO DEL SISTEMA

Registrar clientes, técnicos, tickets, órdenes, asignaciones e intervenciones; consultar el estado de cada caso y conservar sus registros en MySQL entre ejecuciones.

2.5. ACTORES Y REQUERIMIENTOS DEL PROTOTIPO

Actor	Función
Operador / Recepción	Da de alta clientes y tickets.
Coordinador técnico	Genera órdenes y asigna responsables.
Técnico	Realiza trabajos y carga intervenciones.

ID	Requerimiento funcional	Operación de menú
RF01	Registrar y consultar clientes.	1
RF02	Registrar técnicos.	2
RF03	Crear y consultar tickets de distintas categorías.	3, 4
RF04	Generar una orden desde un ticket.	5
RF05	Programar fecha y asignar técnico.	6

ID	Requerimiento funcional	Operación de menú
RF06	Registrar intervenciones.	7
RF07	Cerrar orden y ticket.	8
RF08	Consultar órdenes e historial.	9, 12
RF09	Buscar ticket por identificador.	10
RF10	Ordenar tickets según prioridad.	11

3. APLICACIÓN DEL PROCESO UNIFICADO DE DESARROLLO (PUD)

El PUD organiza el trabajo de forma iterativa. En SIGETEC, el desarrollo comienza por la comprensión del proceso real y continúa con modelos, código y pruebas. El prototipo no equivale a todo el sistema final: es una entrega funcional que puede ampliarse en próximas iteraciones.

Fase	Artefactos y resultados de SIGETEC
Inicio	Problema, actores, objetivo, límites y alcance del prototipo.
Elaboración	Clases del dominio, relaciones entre entidades, DER, arquitectura y reglas de negocio.
Construcción	Código Java, menú de consola, clases POO, JDBC y creación del esquema MySQL.
Transición / pruebas	Compilación y pruebas de lógica realizadas; validación integral de MySQL y evidencia de uso pendientes de realizar localmente.

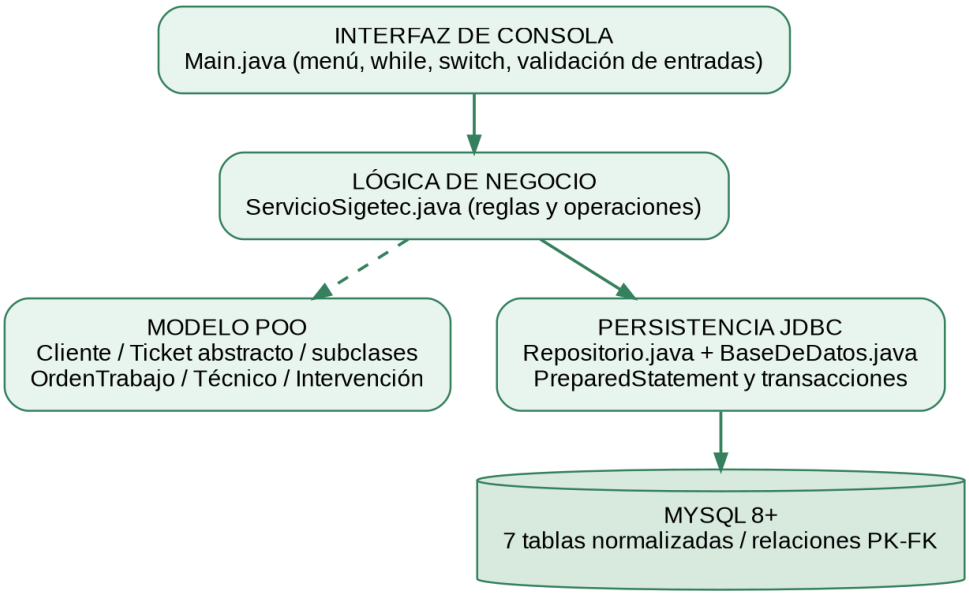


Figura 1. Arquitectura modular utilizada para el prototipo SIGETEC.

Se diferencia la interacción del usuario (menú), la lógica de negocio, el repositorio de datos y MySQL. Esta separación permite ampliar más adelante la interfaz a una aplicación web sin rehacer las reglas del dominio.

4. DESARROLLO JAVA: PROGRAMACIÓN ORIENTADA A OBJETOS

4.1. HERRAMIENTAS Y ESTRUCTURA

El proyecto se desarrolla en Java 17 o superior y se organiza como proyecto Maven compatible con NetBeans. El controlador de base de datos es MySQL Connector/J. Se utilizaron paquetes separados para modelo, persistencia, servicios y consola.

```
ar.edu.siglo21.sigetec
  modelo/    Cliente, Tecnico, Ticket y subclases
  datos/     BaseDeDatos, Repositorio
  servicio/  ServicioSigetec
  consola/   Main
```

4.2. ENCAPSULAMIENTO

Encapsular significa proteger los datos de un objeto. Por ejemplo, nombre y teléfono son atributos privados de Cliente. El constructor revisa que sean correctos y los métodos get permiten leerlos sin exponerlos para una modificación libre.

```
private final String nombre;
public String getNombre() { return nombre; }
```

4.3. HERENCIA Y ABSTRACCIÓN

La clase abstracta Ticket contiene atributos comunes como ID, cliente, prioridad y descripción. TicketCctv y TicketSoporteIt heredan esos atributos; de este modo no es necesario programar nuevamente lo mismo para cada tipo de solicitud.

```
public abstract class Ticket {
   public abstract String instruccionTecnica();
}
public class TicketCctv extends Ticket { /* ... */ }
```

4.4. POLIMORFISMO

El polimorfismo permite utilizar una referencia Ticket para distintos tipos concretos de solicitud. Cuando se invoca instruccionTecnica(), cada subclase responde de manera diferente: CCTV propone revisar la cámara y el enlace de red, mientras que Soporte IT propone diagnosticar el equipo.

```
Ticket t = FabricaTickets.crear(...);
System.out.println(t.instruccionTecnica());
```

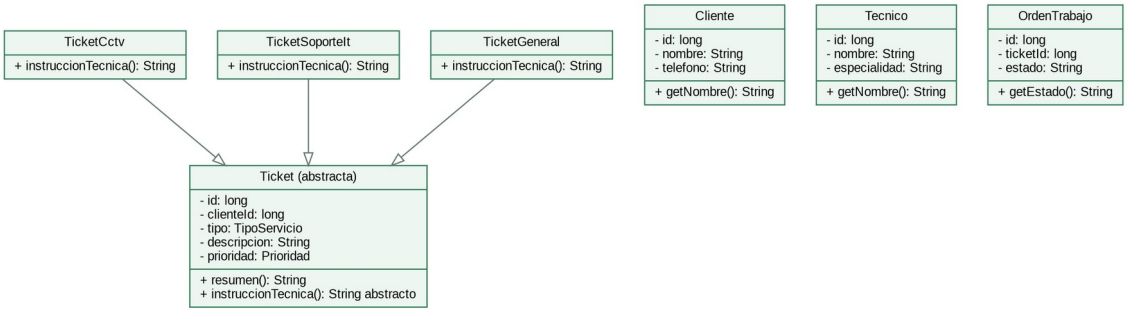


Figura 2. Principales clases del modelo Java y especialización de Ticket.

4.5. CONSTRUCTORES, TIPOS Y ESTRUCTURAS DE CONTROL

Los constructores inicializan objetos con sus valores obligatorios. Se utilizan long para identificadores, String para textos, LocalDate para fechas, boolean para condiciones y enum para estados o prioridades. El menú utiliza un while para permanecer activo y un switch para elegir la acción. Además hay condicionales if para validar datos antes de enviarlos a MySQL.

```
while (continuar) {
   int opcion = leerNumero("Opción: ", 0, 12);
   switch (opcion) {
      case 1 -> registrarCliente();
      case 0 -> continuar = false;
      default -> System.out.println("Opción inválida");
   }
}
```

4.6. MANEJO DE EXCEPCIONES

Para evitar interrupciones del programa, se detectan errores como letras donde debe ingresarse un número, campos vacíos, IDs inexistentes y fallas de acceso a MySQL. El programa muestra una explicación para el usuario y permite continuar con el menú. Las operaciones que modifican varios registros utilizan transacciones con commit y rollback.

```
try {
   servicio.cerrarOrden(id);
} catch (SQLException ex) {
   System.out.println("Error en la base: " + ex.getMessage());
}
```

4.7. ALGORITMOS DE BÚSQUEDA Y ORDENACIÓN

La búsqueda por ID se realiza con SELECT ... WHERE id_ticket = ?, aprovechando la clave primaria de MySQL. La ordenación por prioridad recupera tickets y los organiza mediante Comparator, mostrando primero los de prioridad ALTA, después MEDIA y por último BAJA.

```
tickets.sort(Comparator
   .comparingInt((Ticket t) -> t.getPrioridad().getPuntaje())
   .reversed().thenComparingLong(Ticket::getId));
```

4.8. MENÚ DE SELECCIÓN

El menú dispone de doce operaciones más la salida. El flujo de demostración principal es: cliente → técnico → ticket → orden → planificación/asignación → intervención → cierre. Esta secuencia utiliza la creación de objetos, las reglas de negocio y la persistencia.

[CAPTURA PENDIENTE: menú principal visible después de conectar MySQL]

5. BASE DE DATOS MYSQL Y PERSISTENCIA

5.1. MODELO RELACIONAL Y NORMALIZACIÓN

La base de datos se denomina sigetec y utiliza el motor InnoDB. Se definieron siete tablas: clientes, tecnicos, tipos_servicio, tickets, ordenes_trabajo, asignaciones e intervenciones. Las claves primarias identifican cada registro y las foráneas evitan relaciones inválidas.

Primera Forma Normal: cada campo contiene un dato individual; no se crean columnas tecnico1, tecnico2, etc. Segunda Forma Normal: los atributos no clave dependen de toda su clave primaria; la tabla asignaciones solo agrega la fecha de la relación. Tercera Forma Normal: nombres de técnicos, clientes y tipos de servicio se guardan una sola vez en su propia tabla en lugar de repetirse dentro de cada orden.

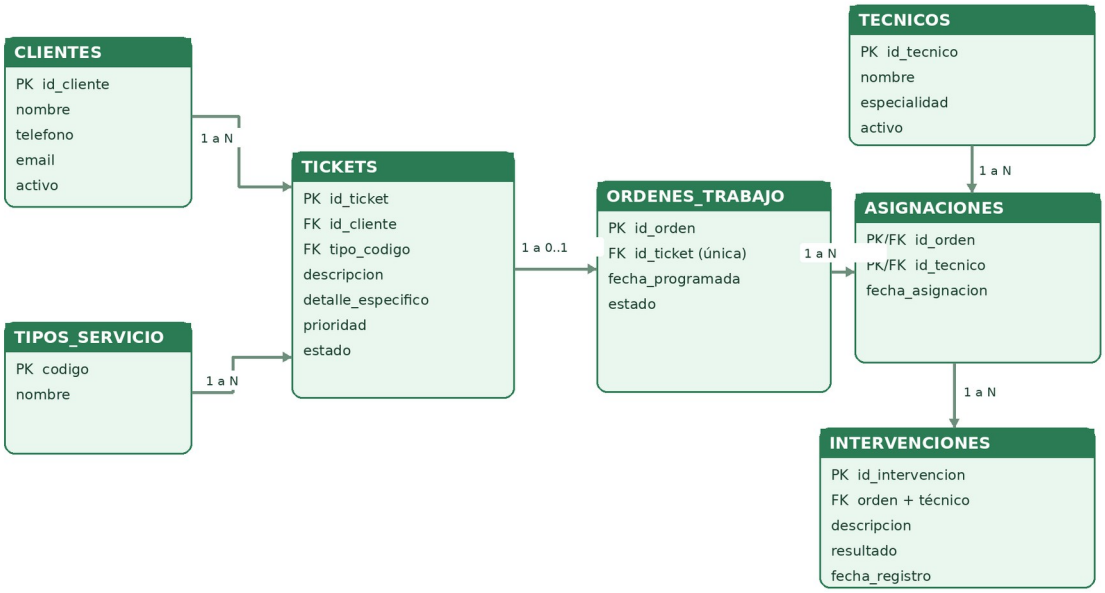


Figura 3. DER del prototipo con claves primarias (PK), foráneas (FK) y cardinalidades.

Por decisión de alcance, cada ticket genera como máximo una orden en este prototipo (UNIQUE en id_ticket). Una orden puede tener varios técnicos y un técnico puede participar en varias órdenes; esta relación se resuelve con asignaciones. La clave foránea compuesta de intervenciones permite verificar que el técnico efectivamente esté asignado a esa orden.

5.2. CREACIÓN DE TABLAS

El archivo sql/01_crear_base.sql crea la base y las siete tablas. El archivo sql/02_datos_referencia.sql carga las categorías iniciales. El usuario debe ejecutar ambos scripts en Workbench antes de iniciar el programa Java.

```
CREATE TABLE asignaciones (
  id_orden BIGINT NOT NULL,
  id_tecnico BIGINT NOT NULL,
  fecha_asignacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY(id_orden, id_tecnico),
  FOREIGN KEY(id_orden) REFERENCES ordenes_trabajo(id_orden),
  FOREIGN KEY(id_tecnico) REFERENCES tecnicos(id_tecnico)
);
```

5.3. CONEXIÓN JDBC Y CONSULTAS PARAMETRIZADAS

BaseDeDatos lee la configuración desde config/sigetec.properties y conecta con DriverManager. Repositorio utiliza PreparedStatement para enviar valores a MySQL sin construir cadenas SQL a partir del texto ingresado por un usuario. Se usa try-with-resources para cerrar conexiones y sentencias automáticamente.

```
String sql = "INSERT INTO clientes (nombre, telefono, email) VALUES (?, ?, ?)";
try (PreparedStatement ps = con.prepareStatement(sql)) {
   ps.setString(1, cliente.getNombre());
   ps.setString(2, cliente.getTelefono());
   ps.setString(3, cliente.getEmail());
   ps.executeUpdate();
}
```

5.4. INSERCIÓN, CONSULTA, ACTUALIZACIÓN Y BORRADO

El repositorio incorpora operaciones INSERT, SELECT y UPDATE necesarias para el funcionamiento del prototipo. Además, el script 04_ejemplo_insert_update_delete.sql demuestra INSERT, UPDATE y DELETE

sobre un cliente ficticio dentro de una transacción que se revierte con ROLLBACK. No se eliminan físicamente tickets con historial asociado, para preservar la integridad referencial.

[CAPTURA PENDIENTE: tablas creadas en MySQL Workbench y resultado de las consultas]

6. ARQUITECTURA, ALGORITMOS Y COMUNICACIONES

La implementación mantiene una arquitectura modular cliente-servidor: el usuario interactúa con la aplicación Java de consola; Java se comunica con MySQL mediante JDBC sobre TCP/IP. El puerto habitual es 3306, modificable según la instalación local del servidor. En la práctica académica se utiliza localhost, sin abrir el servidor de base de datos a Internet.

Elemento	Definición elegida
Cliente	Aplicación Java de consola ejecutada en NetBeans.
Capa de negocio	ServicioSigetec con validaciones y reglas.
Capa de datos	Repositorio y MySQL Connector/J (JDBC).
Base de datos	MySQL 8.0+ / InnoDB / puerto TCP 3306 local.
Red y transporte	TCP/IP sobre la interfaz de red local. En red física, Ethernet IEEE 802.3.
Seguridad	Credenciales fuera del repositorio; PreparedStatement; usuario MySQL con permisos limitados. Para una futura aplicación remota se deberá exigir TLS.

La separación entre presentación, lógica y datos facilita que el sistema crezca sin cambiar todas sus partes a la vez. El prototipo no implementa autenticación de usuarios ni acceso remoto a MySQL; esas capacidades son trabajo futuro y no se presentan como implementadas.

7. ETAPA DE PRUEBAS Y TRAZABILIDAD

7.1. PRUEBAS DE MODELO EJECUTADAS

Se compiló el código fuente con OpenJDK 21 usando compatibilidad Java 17 y se ejecutaron 16 comprobaciones automatizadas independientes de MySQL. Todas finalizaron correctamente. Las pruebas cubren constructores, atributos privados, validaciones, creación de subclases de Ticket, polimorfismo, búsqueda y ordenación.

Tipo	Validación	Resultado observado
Compilación	javac --release 17 sobre archivos .java	Aprobado en entorno de preparación
Modelo POO	16 pruebas en PruebasModelo.java	16/16 aprobadas
Persistencia MySQL	Conexión JDBC y operaciones sobre el servidor local	Pendiente de ejecutar en computadora del alumno
Demostración de usuario	Registro, cierre y reapertura del programa	Pendiente de captura real

7.2. PLAN DE PRUEBA PARA LA EJECUCIÓN LOCAL

CP	RF/CU	Procedimiento	Resultado esperado
CP01	RF01	Registrar un cliente con datos válidos.	MySQL guarda cliente con nuevo ID.
CP02	RF03	Crear ticket CCTV asociado al cliente.	Ticket guardado con prioridad y estado.
CP03	RF04	Generar orden desde el ticket.	Orden pendiente; ticket en atención.
CP04	RF05	Programar y asignar un técnico.	Orden programada y asignación persistidas.
CP05	RF06-RF07	Registrar intervención y finalizar.	Orden finalizada y ticket cerrado.
CP06	RF08	Salir y abrir nuevamente la aplicación.	Persistencia de los registros demostrada.
CP07	RF09	Buscar ID inexistente.	Mensaje informativo; no se detiene el menú.
CP08	RF10	Crear tickets de prioridades variadas.	Orden ALTA, MEDIA, BAJA.

7.3. TRAZABILIDAD

RF	Clase/método	Tabla	Caso de prueba
RF01	ServicioSigetec.registrarCliente	clientes	CP01
RF03	FabricaTickets + Repositorio.registrarTicket	tickets	CP02
RF04	Repositorio.crearOrden	ordenes_trabajo	CP03
RF05	Repositorio.programarYAsignar	ordenes_trabajo/ asignaciones	CP04
RF06-07	Repositorio.registrarIntervencion/ finalizarOrden	intervenciones/tickets	CP05
RF08	Repositorio.listarOrdenes	consultas JOIN	CP06
RF09-10	buscarTicket / ticketsPorPrioridad	tickets	CP07-08

[CAPTURA PENDIENTE: registro real en consola: cliente, ticket, orden, técnico y finalización]

[CAPTURA PENDIENTE: Workbench: SELECT posteriores al reinicio de SIGETEC]

8. EJECUCIÓN EN NETBEANS Y GITHUB

El proyecto se entrega con un archivo pom.xml para abrirlo como proyecto Maven en NetBeans. También incluye scripts SQL y el archivo de ejemplo para la configuración local. Las credenciales reales se mantienen en un archivo privado excluido por .gitignore.

6. Instalar JDK 17+, MySQL Server 8+, MySQL Workbench y NetBeans.
7. Ejecutar sql/01_crear_base.sql y sql/02_datos_referencia.sql en Workbench.
8. Crear o seleccionar un usuario MySQL y completar config/sigetec.properties.
9. Abrir la carpeta del proyecto desde NetBeans, realizar Clean and Build y luego Run Project.
10. Ejecutar el recorrido completo del menú y comprobar MySQL con sql/03_consultas.sql.
11. Guardar capturas reales de resultados y agregar al informe la URL pública del repositorio de GitHub.

Enlace del repositorio GitHub: [PENDIENTE DE PUBLICAR POR EL ALUMNO]. La entrega del archivo ZIP no equivale a subirlo a GitHub. Antes de entregar es necesario añadir aquí la URL real, verificando que estén disponibles el código y los scripts asociados.

9. CONCLUSIÓN

SIGETEC permite demostrar, a escala de prototipo, que el circuito de atención de un servicio técnico puede documentarse a través de objetos Java y registros persistentes en un modelo relacional. La separación en capas, los conceptos de POO, la validación de datos y el manejo de transacciones sientan una base organizada para las próximas iteraciones del sistema.

La compilación y las pruebas automatizadas del modelo fueron verificadas en el entorno de preparación. Para finalizar la evaluación práctica todavía corresponde ejecutar el prototipo contra una instalación real de MySQL, documentar las operaciones y publicar la versión completa en GitHub.

10. REFERENCIAS

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas. Pearson Educación.

Oracle. (s. f.). Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/

Oracle. (s. f.). MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/

Oracle. (s. f.). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/